

Day 10

Optimizations

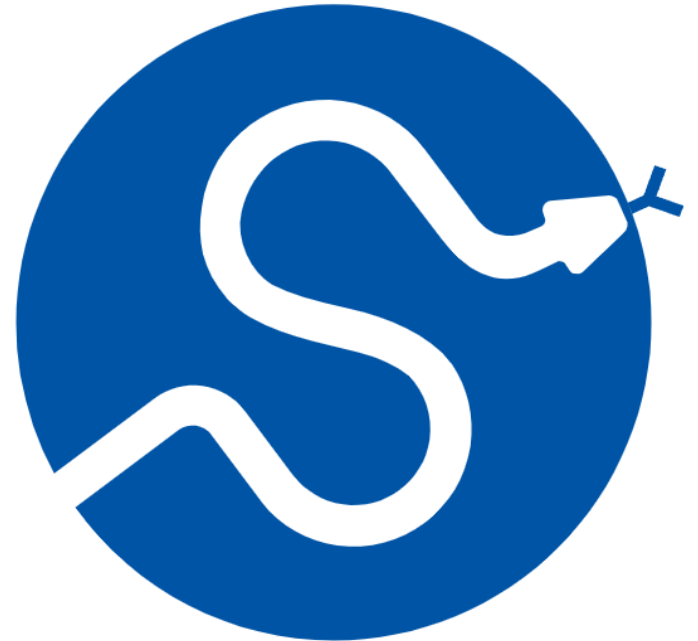
Lecturer: Msc. Minh Tan Le

Today's lesson includes...

- I. What is SciPy?
- II. SciPy features

Introduction

- SciPy is a Python library for scientific programming.
- Built on top of Numpy, but focus on mathematical algorithms and convenience functions.



Features

- `scipy.integrate` (graph areas, volumes) 🧐
- `scipy.optimize` (min, max) 🧐
- `scipy.interpolate` (curve fitting, prediction) 🧐
- `scipy.signal` (signal processing)
- `scipy.stats` (statistics)

scipy.integrate

- Quad: General purpose integration.
- Dblquad: Multi-variable integration.

$$\int_{y=0}^1 \int_{x=0}^1 xy dx dy$$

scipy.optimize

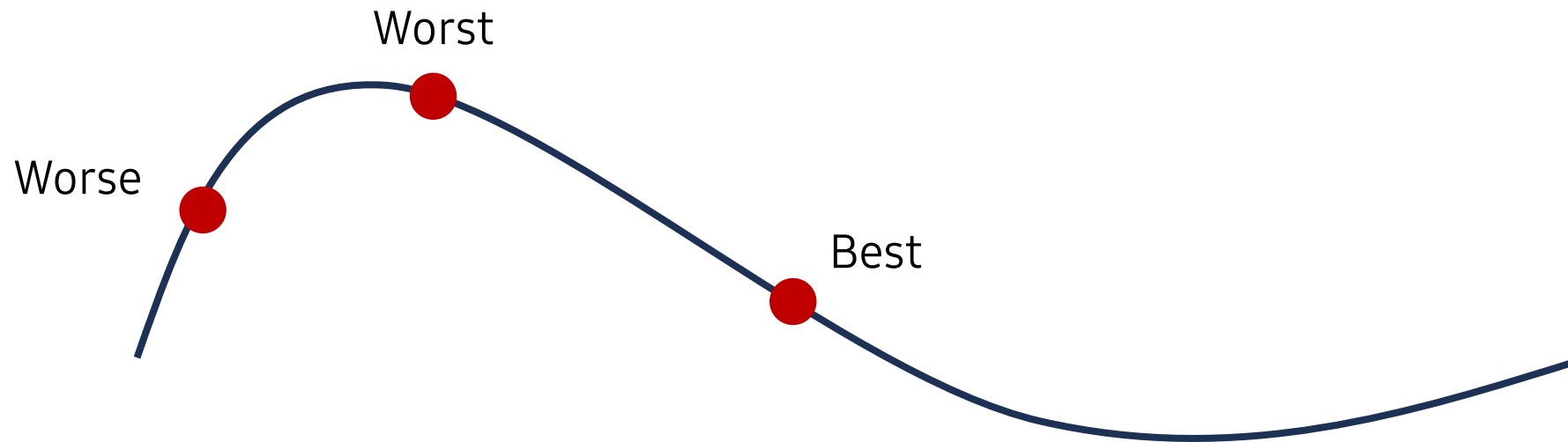
- Black box
 - Nelder-Mead simplex 
- White box
 - Broyden-Fletcher-Goldfarb-Shanno 
 - Newton-Conjugate-Gradient

$$f(x^*) \leq f(x), x \approx x^*$$

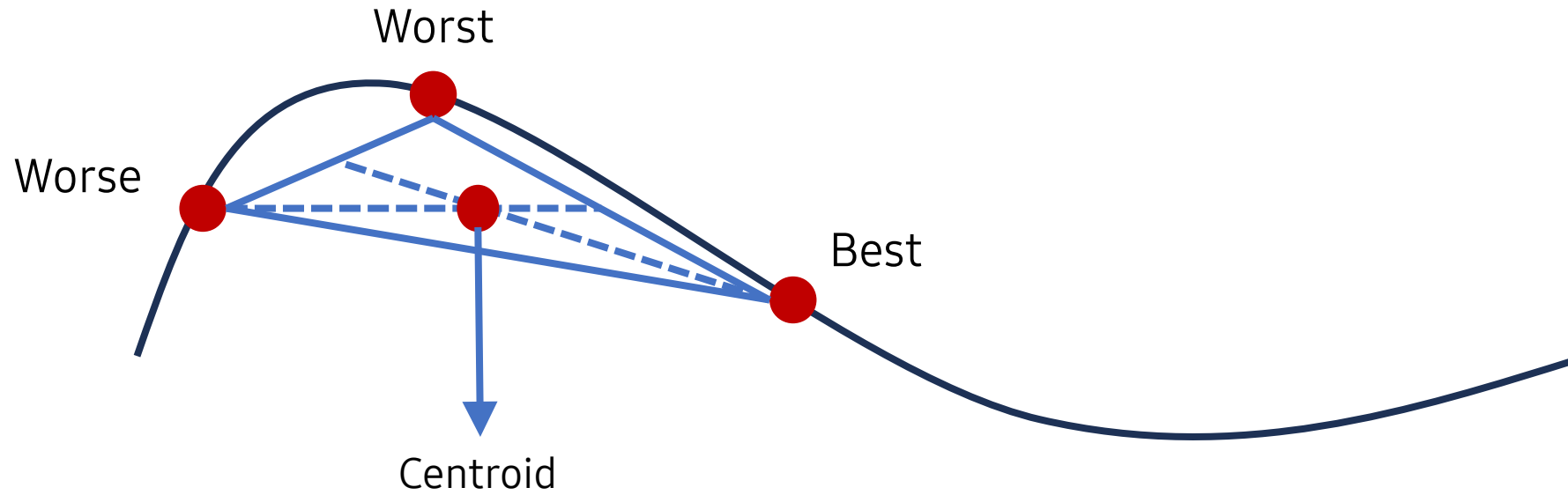
$$\operatorname{argmin}(f(x))$$

Nelder-Mead simplex

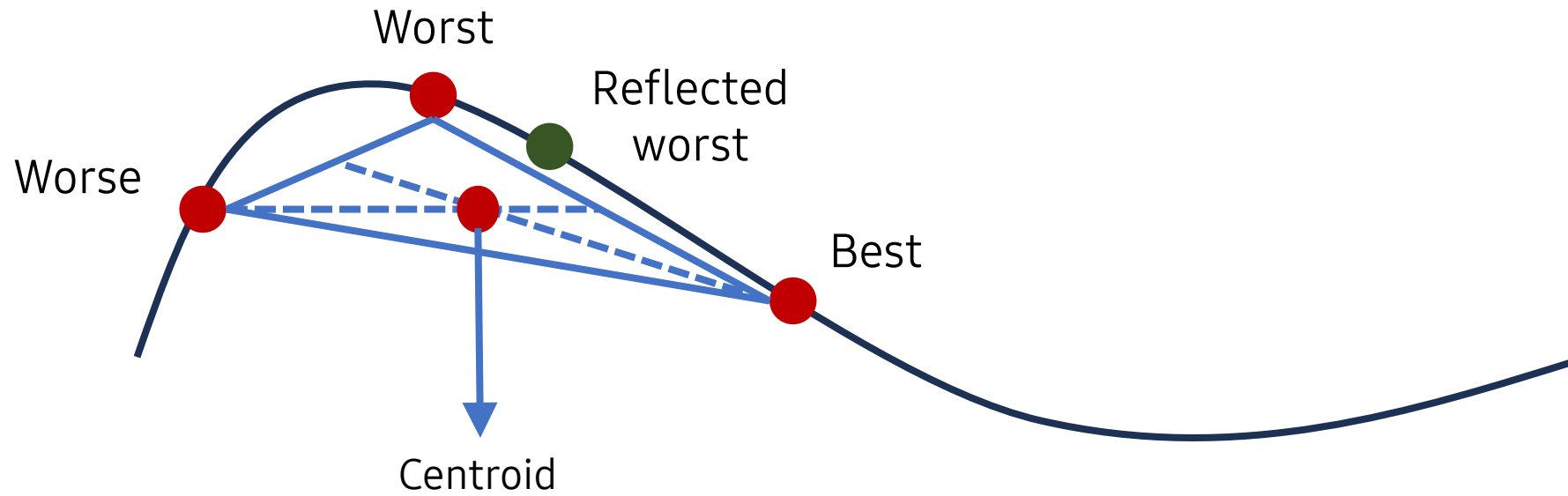
- Idea: We get start with three initial x (simplex).
- A simplex can be interpreted as a triangle.



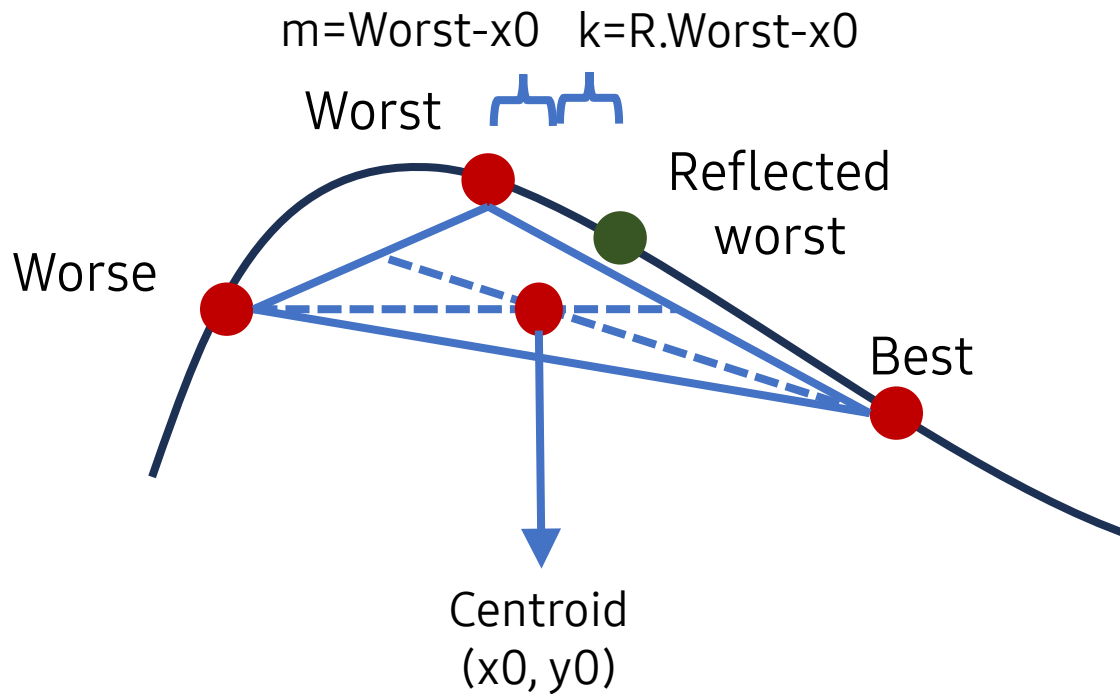
#2. Find the centroid



#3. Reflection

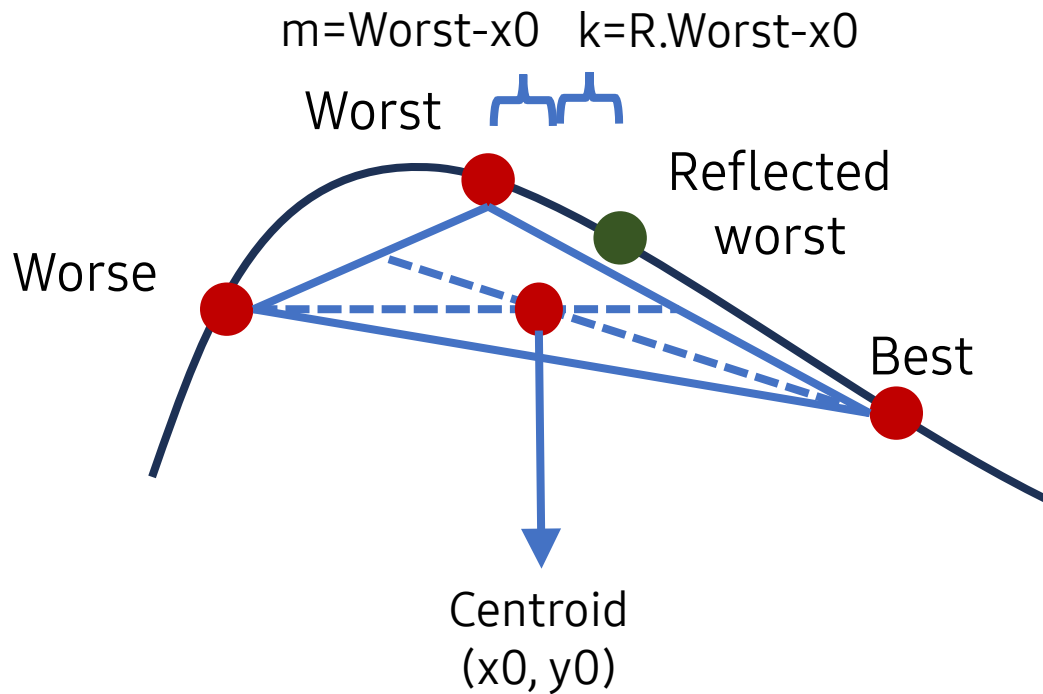


#4. Reflection, Expansion & Contraction



```
Assuming R.Worst = x0 + k
If f(R.Worst) < f(Worse):
    if f(R.Worst) >= f(Best):
        Worst = R.Worst
    else:
        E.Worst = x0 + 2k
        if f(E.Worst) < f(R.Worst):
            Worst = E.Worst
        else:
            Worst = R.Worst
else:
    if f(R.Worst) < f(Worst):
        C.Worst = x0 + 1/2 k
        if f(C.Worst) < f(R.Worst):
            Worst = C.Worst
        else:
            Simplex → Best
```

#4. Reflection, Expansion & Contraction



else:

$$C.Worst = x0 + \frac{1}{2} m$$

if $f(C.Worst) < f(Worst)$:

$Worst = C.Worst$

else:

 Simplex $\rightarrow$ Best

Def Simplex $\rightarrow$ Best (**Shrink**):

for each x in $[Worst, Worse]$:

$$x = Best + \frac{1}{2}(x - Best)$$

Broyden-Fletcher-Goldfarb-Shanno

- Before we begin, let's examine this function:

$$f(x) = x^3 - 3x + 2$$

$$f'(x) = 3x^2 - 3$$

x	x	-1		1	x
x	+	0	-	0	+
x		4			
$f'(x)$				0	



Broyden-Fletcher-Goldfarb-Shanno

- Before we begin, let's examine this function:

$$f(x) = x^3 - 3x + 2$$

$$f'(x) = 3x^2 - 3$$

x	x	-1		1	x
x	+	0	-	0	+
x		4			
$f'(x)$				0	

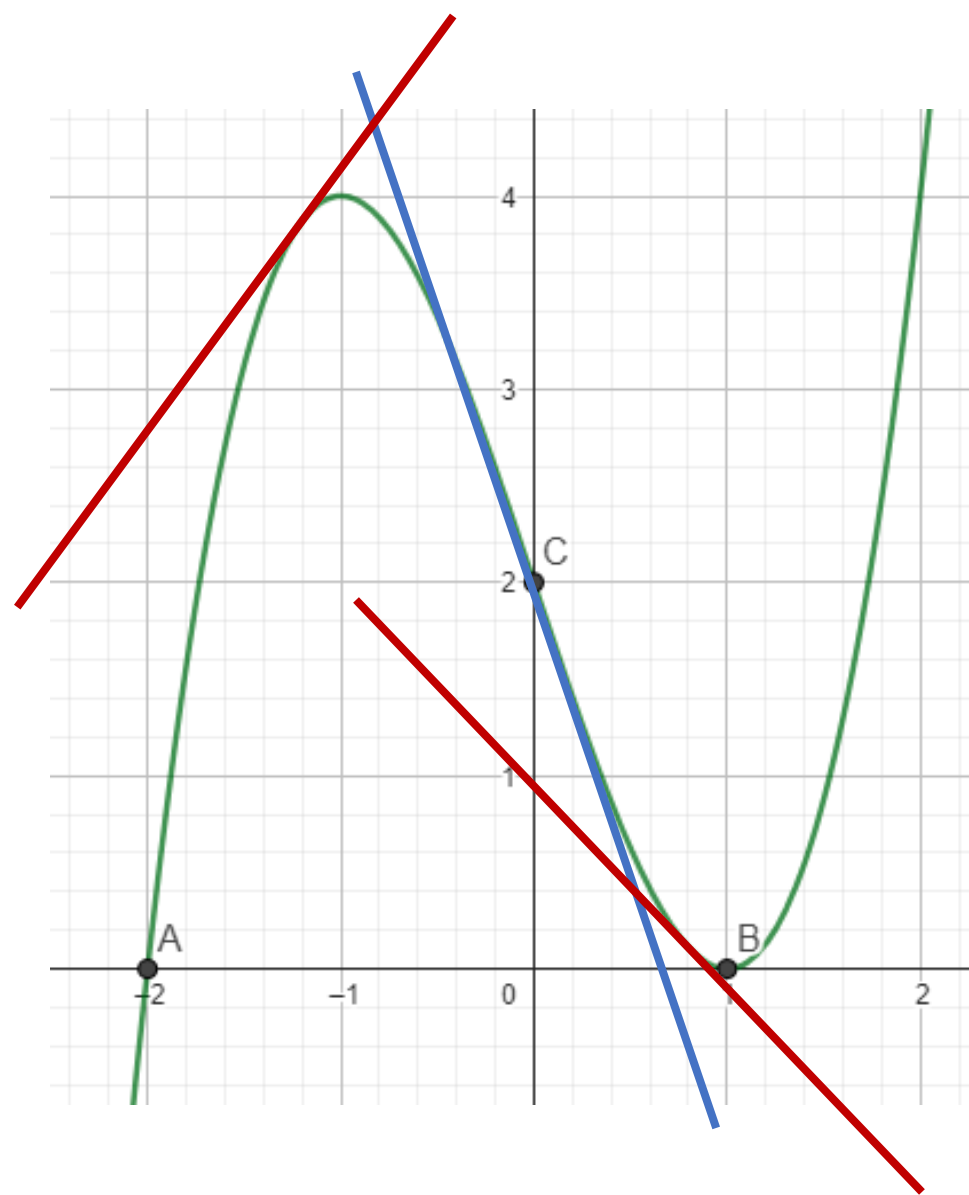


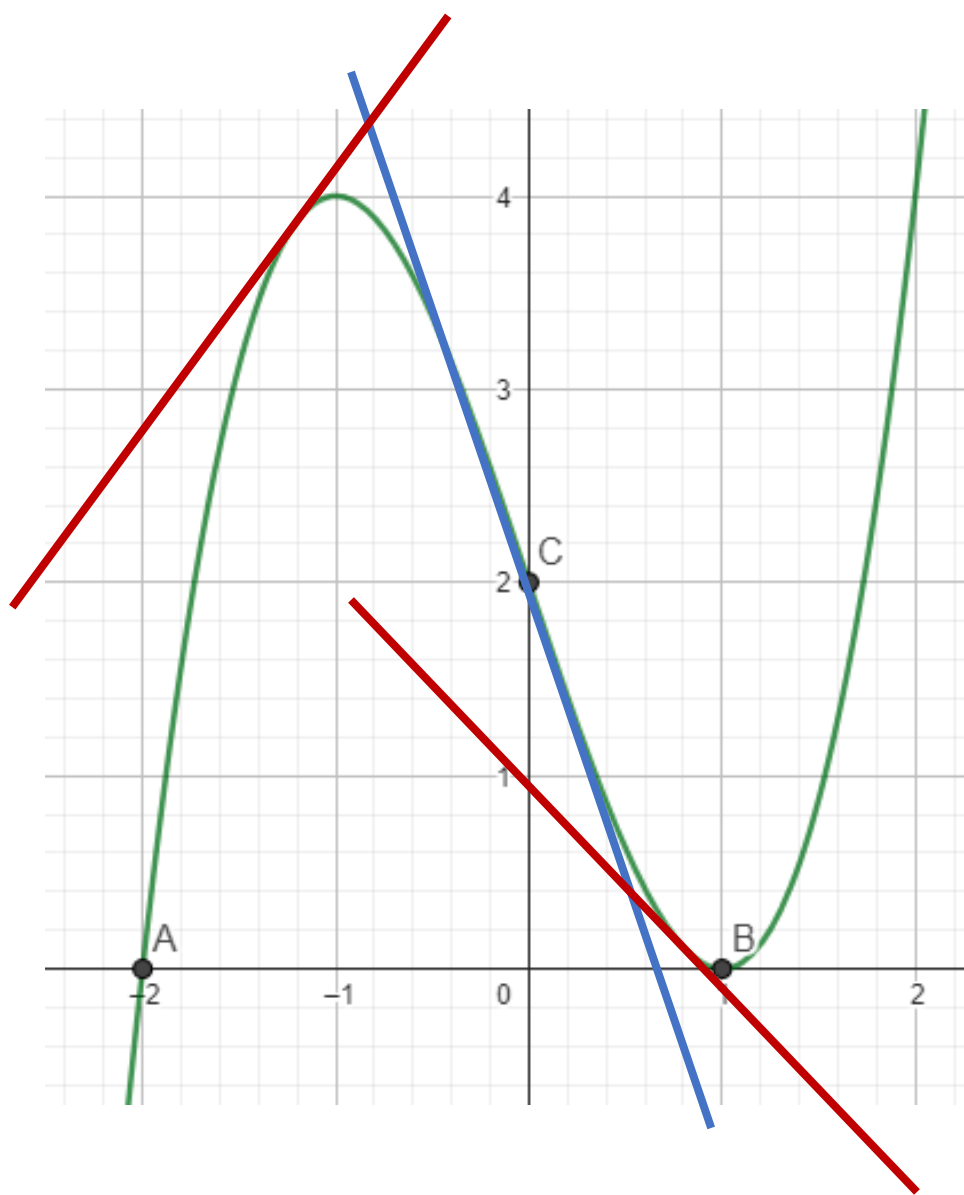
Critical points/
Stationary point

Inflection points

$$f''(x) = 6x = 0$$

(0,2)





$$m(x|x_0) = f'(x_0)x + f(x_0) - x_0f'(x_0)$$

Hessian matrix

$$f : R^n \rightarrow R$$

$$H_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & & & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

$$H_f = \left[\frac{\partial^2 f}{\partial x^2} \right] = [6x] = H_f(x)$$

$$f(x, y) = x^3 + 2y^2 + 3xy$$

$$H_f = \begin{bmatrix} 6x & 3 \\ 3 & 4 \end{bmatrix}$$

$$\text{Det}(H_f) = \begin{vmatrix} 6x & 3 \\ 3 & 4 \end{vmatrix} = 6x \cdot 4 - 3 \cdot 3 = 24x - 9$$

$$f(x, y) = x^3 + 2y^2 + 3xy^2$$

$$H_f = \begin{bmatrix} \dots(1) & \dots(2) \\ \dots(3) & \dots(4) \end{bmatrix}$$

$$\text{Det}(H_f) = \dots = 36x^2 + 24x - 36y^2$$

More about H matrix

- If H is **positive-definite** (eigenvalues are positive), the critical point is **local minimum**.
- If H is **negative-definite** (eigenvalues are negative), the critical point is **local maximum**.
- If H is **indefinite** (eigenvalues are mixed), the critical point is an **inflection point**.

simpler: Second partial derivative test

- If $\text{Det}(a, b) > 0$,
 - If $f_{xx}(a, b) = \frac{\partial^2 f}{\partial x^2}(a, b) > 0$, critical point (a, b) is a local minimum.
 - If $f_{xx}(a, b) = \frac{\partial^2 f}{\partial x^2}(a, b) < 0$, critical point (a, b) is a local maximum.
- If $\text{Det}(a, b) < 0$, critical point is an inflection point.

Broyden-Fletcher-Goldfarb-Shanno

$$p_k = -H_k \nabla f(x_k)$$

$$\alpha = \operatorname{argmin}(f(x_k + \alpha p_k))$$

$$s_k = \alpha_k p_k$$

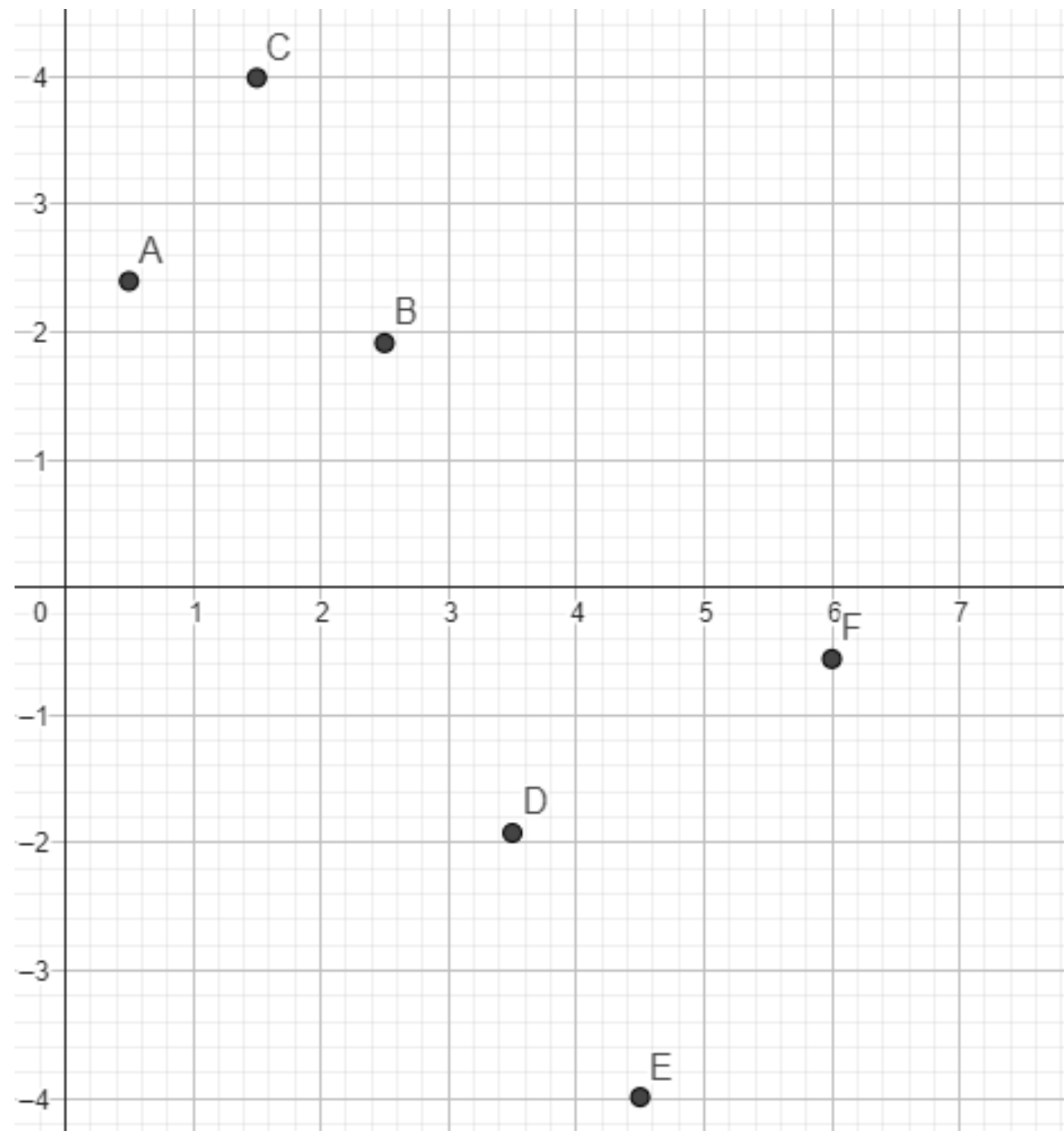
$$x_{k+1} = x_k + s_k$$

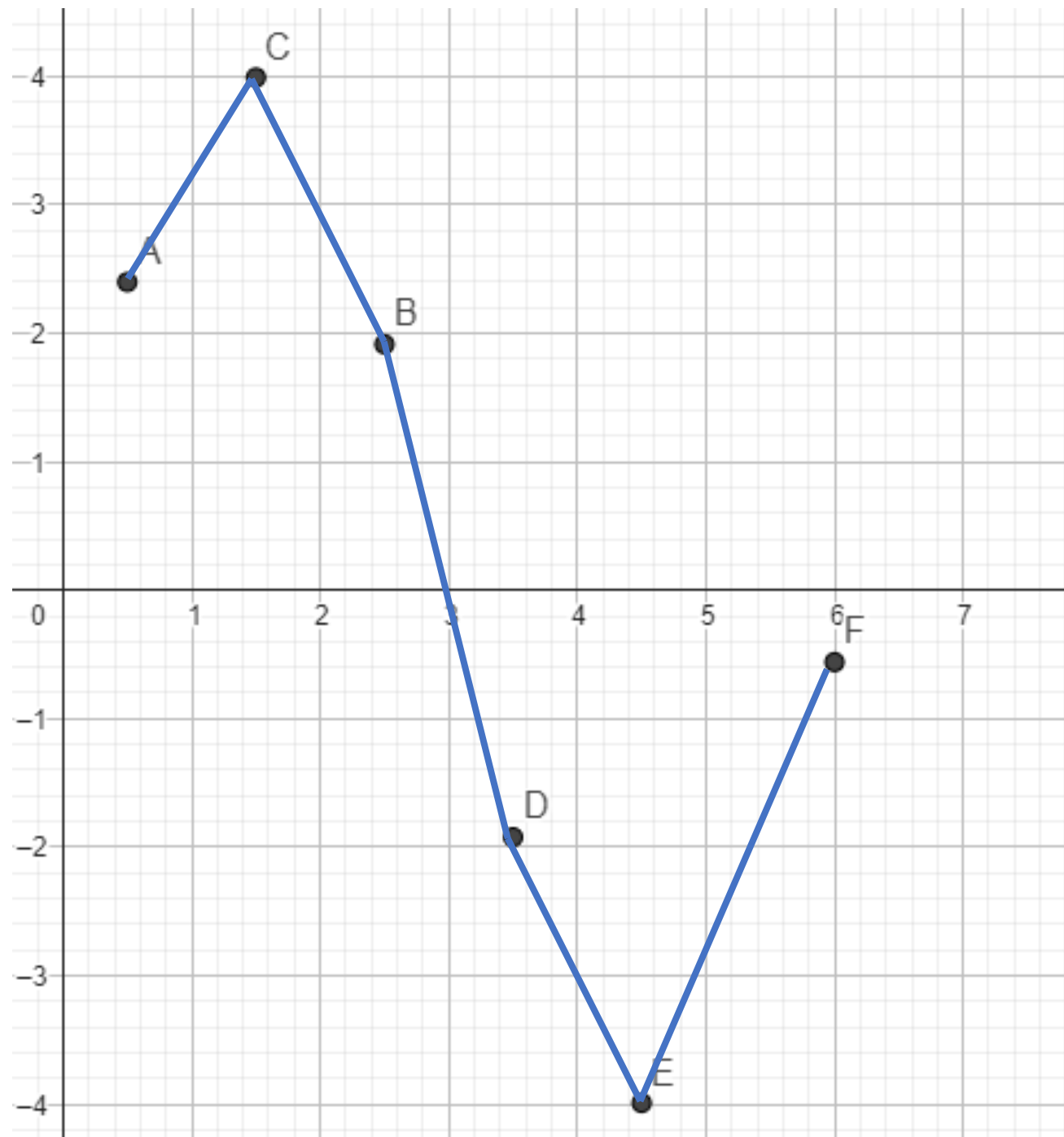
$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

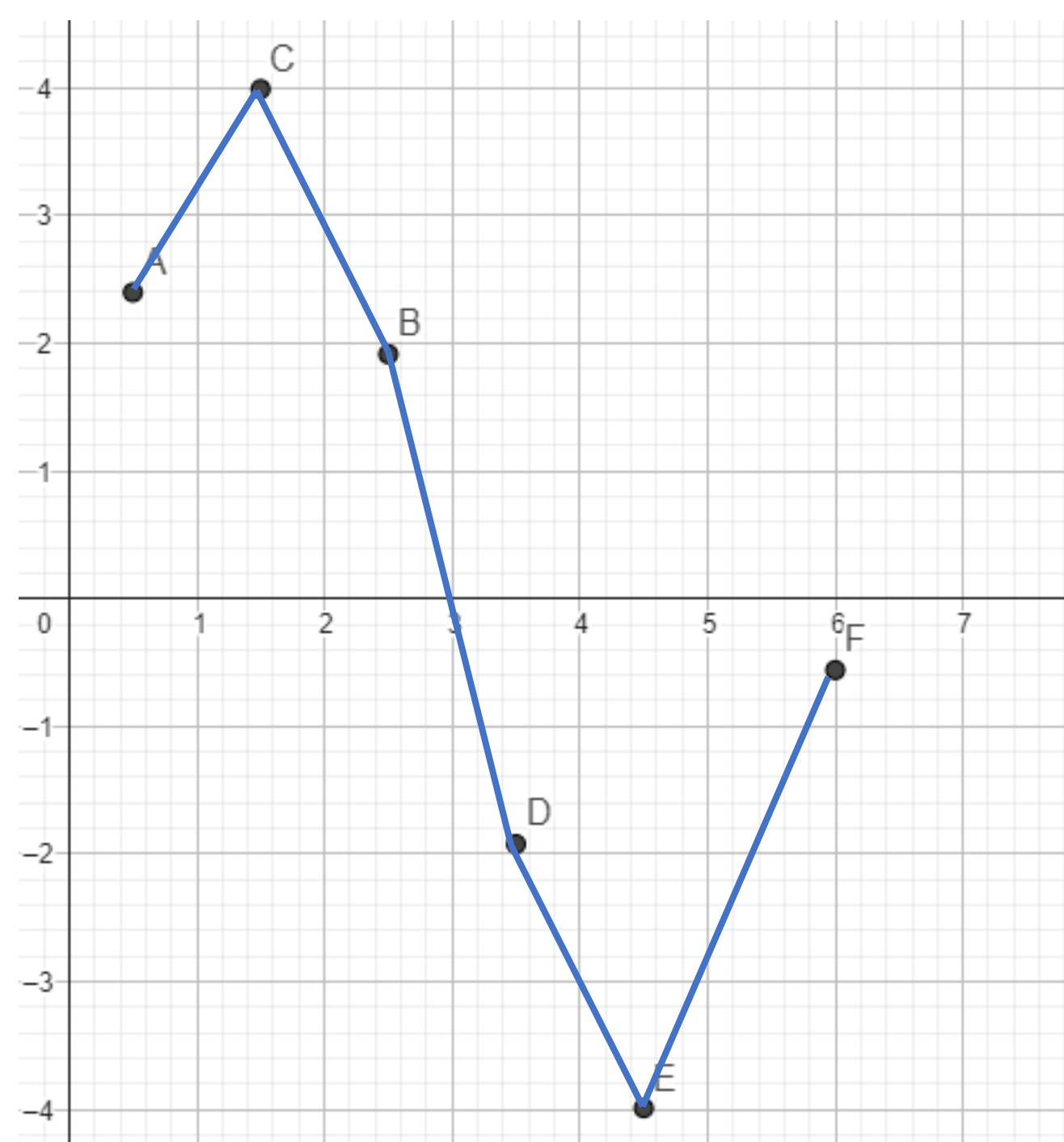
$$H_{k+1} = H_k + \frac{(\mathbf{s}_k^T \mathbf{y}_k + \mathbf{y}_k^T H_k \mathbf{y}_k)(\mathbf{s}_k \mathbf{s}_k^T)}{(\mathbf{s}_k^T \mathbf{y}_k)^2} - \frac{H_k \mathbf{y}_k \mathbf{s}_k^T + \mathbf{s}_k \mathbf{y}_k^T H_k}{\mathbf{s}_k^T \mathbf{y}_k}$$

scipy.interpolate

x	y
0.5	2.40
1.5	3.99
2.5	1.91
3.5	-1.92
4.5	-3.99
6.0	-0.56

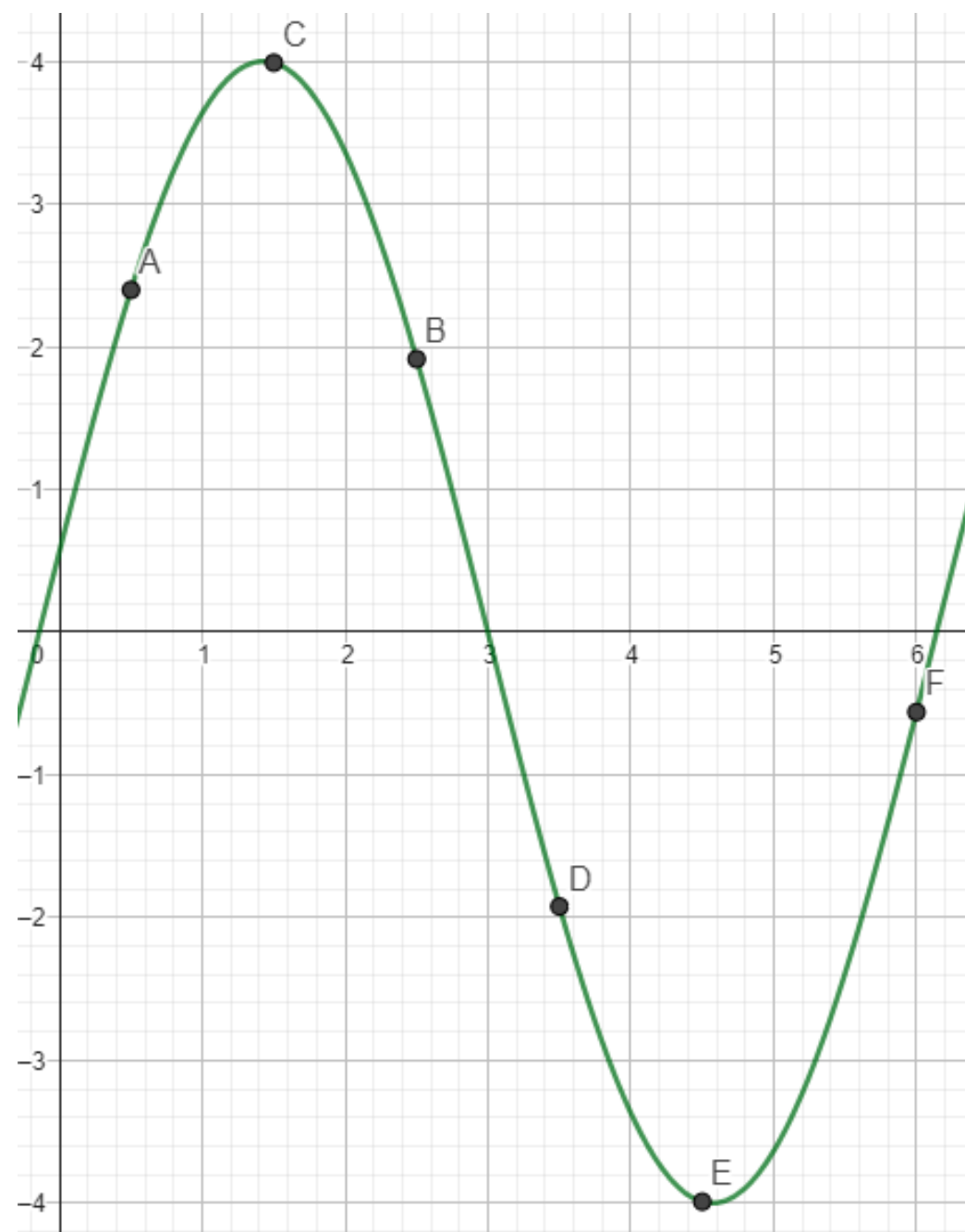






Linear interpolation:

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0}$$



Polynomial interpolation (general):

$$y = \sum_{i=1}^{k \leq n-1} C_i x^i$$

$$C \in R^{k \leq n-1}$$

Theorem: if there are n data points, there is exactly one polynomial of degree at most $n-1$ going through all points.

x	y
0.5	2.40
1.5	3.99
2.5	1.91
3.5	-1.92
4.5	-3.99
6.0	-0.56

Lagrange polynomial interpolation:

Step 1. Define a POS accepting data as root:

$$\begin{aligned}
 g(x) &= \prod_{i=0} (x - x_i) \\
 &= (x - 0.5)(x - 1.5)(x - 2.5)(x - 3.5)(x - 4.5)(x - 6.0)
 \end{aligned}$$

x	y
0.5	2.40
1.5	3.99
2.5	1.91
3.5	-1.92
4.5	-3.99
6.0	-0.56

Lagrange polynomial interpolation:

Step 2. Calculate weights:

$$w_j = \left(\prod_{i \neq j} x_j - x_i \right)^{-1}$$

$$\begin{aligned}
 w_0 &= ((0.5 - 1.5) \cdot (0.5 - 2.5) \dots)^{-1} \\
 &= \frac{-1}{122}
 \end{aligned}$$

Lagrange polynomial interpolation:

Step 3. Find $g_j(x)$

$$g_j(x) = \frac{w_j \cdot y_j \cdot g(x)}{x - x_j}$$

x	y	x
0.5	2.40	x
		0.5
1.5	3.99	x
		0.5
2.5	1.91	x
		0.5
3.5	-1.92	x
		0.5
4.5	-3.99	x
		0.5
6.0	-0.56	x
		0.5

Lagrange polynomial interpolation:

Step 4. Find Lagrange function:

$$l(x) = \sum_i g_j(x)$$

x	y	x
0.5	2.40	x
		0.5
1.5	3.99	x
		0.5
2.5	1.91	x
		0.5
3.5	-1.92	x
		0.5
4.5	-3.99	x
		0.5
6.0	-0.56	x
		0.5

Exercise (group)

1. Use Zeppelin to **manually** build a Lagrange interpolation from the data.
2. Run Lagrange interpolation **using SciPy** and compare the result.

x	y
1.5	-0.96
2	-2.71
3.5	-3.13
5	2.26
6.5	3.45
7	2.06